

bit_analysis_12

July 10, 2025

1 Análisis de eficiencia de bits 12 metros de distancia

Este notebook analiza qué tan parecidos son los bits recibidos `bits_rx` con los bits transmitidos `bits_tx` y presenta gráficas, métricas de rendimiento y las imágenes que no cumplen con el criterio de coincidencia.

```
[38]: import pandas as pd
from pathlib import Path
from IPython.display import display, Image
import matplotlib.pyplot as plt

path = "./results/12m/"
nombre_csv = "resultados_comparacion_12m.csv"
df = pd.read_csv(path + nombre_csv)

# Vista rápida del DataFrame
display(df.head())
print(f"Filas totales: {len(df)}")
```

	num_auto	bw_plate_tx \
0	1	dataset_yolo/autos_bw/auto001.png
1	1	dataset_yolo/autos_bw/auto001.png
2	1	dataset_yolo/autos_bw/auto001.png
3	1	dataset_yolo/autos_bw/auto001.png
4	1	dataset_yolo/autos_bw/auto001.png

	bw_plate_rx	ocr_txt_tx	bits_equal	bit_errors \
0	data_12m/autos_bw/auto001_rep01.png	K0Z.808	True	0
1	data_12m/autos_bw/auto001_rep02.png	K0Z.808	True	0
2	data_12m/autos_bw/auto001_rep03.png	K0Z.808	True	0
3	data_12m/autos_bw/auto001_rep04.png	K0Z.808	True	0
4	data_12m/autos_bw/auto001_rep05.png	K0Z.808	True	0

	bits_tx	bits_rx
0	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
1	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
2	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
3	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]

```
4 [0 0 1 ... 0 0 0] [0 0 1 ... 0 0 0]
```

Filas totales: 1534

1.1 Distribución de errores de bit por imagen

```
[39]: plt.figure()
df['bit_errors'].plot(kind='hist', bins=30)
plt.xlabel('Errores de bit por imagen')
plt.ylabel('Frecuencia')
plt.title('Histograma de errores de bit')
plt.show()
```



1.2 Errores de bit por réplica

```
[40]: plt.figure(figsize=(10,4))
plt.plot(df.index, df['bit_errors'], marker='o', linestyle='none')
plt.xlabel('Índice de fila')
plt.ylabel('Errores de bit')
plt.title('Errores de bit por réplica')
```

```
plt.show()
```



1.3 Imágenes que no cumplen el criterio

```
[41]: # Eliminamos las que tienen 2000 errores, ya que es la imagen inversa
df = df[df['bit_errors'] < 2000]
```

```
[42]: import numpy as np    # por si no estaba importado

# --- Parámetros de tolerancia ---
BITS_TOTAL = 2000
max_errors = 100 # máximo de bits erróneos permitidos
max_ber    = max_errors / BITS_TOTAL # % BER máximo

# --- BER por fila ---
df['bits_total'] = BITS_TOTAL
df['ber'] = df['bit_errors'] / BITS_TOTAL

# --- Criterio de falla ---
fail_df = df[(df['bit_errors'] > max_errors) | (df['ber'] > max_ber)]

print(f'Imágenes que superan el umbral: {len(fail_df)}')
print(f'Total de imágenes: {len(df)}')
print(f'Porcentaje de fallos: {len(fail_df) / len(df) * 100:.2f}%')
display(
    fail_df[['num_auto', 'bw_plate_tx', 'bw_plate_rx',
              'bit_errors', 'ber']]
)
```

Imágenes que superan el umbral: 53

Total de imágenes: 1529

Porcentaje de fallos: 3.47%

	num_auto	bw_plate_tx \
66	64	dataset_yolo/autos_bw/auto064.png
123	64	dataset_yolo/autos_bw/auto064.png
124	64	dataset_yolo/autos_bw/auto064.png
186	53	dataset_yolo/autos_bw/auto053.png
187	53	dataset_yolo/autos_bw/auto053.png
335	45	dataset_yolo/autos_bw/auto045.png
364	104	dataset_yolo/autos_bw/auto104.png
378	122	dataset_yolo/autos_bw/auto122.png
379	106	dataset_yolo/autos_bw/auto106.png
392	107	dataset_yolo/autos_bw/auto107.png
407	109	dataset_yolo/autos_bw/auto109.png
479	61	dataset_yolo/autos_bw/auto061.png
553	89	dataset_yolo/autos_bw/auto089.png
570	127	dataset_yolo/autos_bw/auto127.png
661	36	dataset_yolo/autos_bw/auto036.png
790	93	dataset_yolo/autos_bw/auto093.png
791	93	dataset_yolo/autos_bw/auto093.png
792	93	dataset_yolo/autos_bw/auto093.png
793	95	dataset_yolo/autos_bw/auto095.png
794	127	dataset_yolo/autos_bw/auto127.png
808	94	dataset_yolo/autos_bw/auto094.png
914	56	dataset_yolo/autos_bw/auto056.png
917	56	dataset_yolo/autos_bw/auto056.png
926	56	dataset_yolo/autos_bw/auto056.png
977	105	dataset_yolo/autos_bw/auto105.png
979	128	dataset_yolo/autos_bw/auto128.png
1131	48	dataset_yolo/autos_bw/auto048.png
1132	48	dataset_yolo/autos_bw/auto048.png
1133	48	dataset_yolo/autos_bw/auto048.png
1134	48	dataset_yolo/autos_bw/auto048.png
1230	33	dataset_yolo/autos_bw/auto033.png
1306	138	dataset_yolo/autos_bw/auto138.png
1307	138	dataset_yolo/autos_bw/auto138.png
1362	32	dataset_yolo/autos_bw/auto032.png
1363	32	dataset_yolo/autos_bw/auto032.png
1364	32	dataset_yolo/autos_bw/auto032.png
1365	16	dataset_yolo/autos_bw/auto016.png
1389	96	dataset_yolo/autos_bw/auto096.png
1390	96	dataset_yolo/autos_bw/auto096.png
1391	96	dataset_yolo/autos_bw/auto096.png
1439	2	dataset_yolo/autos_bw/auto002.png
1440	130	dataset_yolo/autos_bw/auto130.png
1441	138	dataset_yolo/autos_bw/auto138.png
1442	130	dataset_yolo/autos_bw/auto130.png

1449	175	dataset_yolo/autos_bw/auto175.png
1450	175	dataset_yolo/autos_bw/auto175.png
1451	139	dataset_yolo/autos_bw/auto139.png
1462	45	dataset_yolo/autos_bw/auto045.png
1474	46	dataset_yolo/autos_bw/auto046.png
1523	26	dataset_yolo/autos_bw/auto026.png
1524	26	dataset_yolo/autos_bw/auto026.png
1525	26	dataset_yolo/autos_bw/auto026.png
1526	26	dataset_yolo/autos_bw/auto026.png

	bw_plate_rx	bit_errors	ber
66	data_12m/autos_bw/auto064_rep01.png	749	0.3745
123	data_12m/autos_bw/auto064_rep02.png	792	0.3960
124	data_12m/autos_bw/auto064_rep03.png	790	0.3950
186	data_12m/autos_bw/auto053_rep01.png	949	0.4745
187	data_12m/autos_bw/auto053_rep02.png	945	0.4725
335	data_12m/autos_bw/auto045_rep01.png	755	0.3775
364	data_12m/autos_bw/auto104_rep01.png	952	0.4760
378	data_12m/autos_bw/auto122_rep01.png	672	0.3360
379	data_12m/autos_bw/auto106_rep01.png	910	0.4550
392	data_12m/autos_bw/auto107_rep01.png	824	0.4120
407	data_12m/autos_bw/auto109_rep01.png	773	0.3865
479	data_12m/autos_bw/auto061_rep01.png	622	0.3110
553	data_12m/autos_bw/auto089_rep01.png	1032	0.5160
570	data_12m/autos_bw/auto127_rep01.png	1136	0.5680
661	data_12m/autos_bw/auto036_rep10.png	810	0.4050
790	data_12m/autos_bw/auto093_rep01.png	902	0.4510
791	data_12m/autos_bw/auto093_rep02.png	902	0.4510
792	data_12m/autos_bw/auto093_rep03.png	902	0.4510
793	data_12m/autos_bw/auto095_rep01.png	676	0.3380
794	data_12m/autos_bw/auto127_rep02.png	934	0.4670
808	data_12m/autos_bw/auto094_rep01.png	805	0.4025
914	data_12m/autos_bw/auto056_rep16.png	1009	0.5045
917	data_12m/autos_bw/auto056_rep17.png	1008	0.5040
926	data_12m/autos_bw/auto056_rep18.png	1009	0.5045
977	data_12m/autos_bw/auto105_rep13.png	718	0.3590
979	data_12m/autos_bw/auto128_rep01.png	992	0.4960
1131	data_12m/autos_bw/auto048_rep13.png	896	0.4480
1132	data_12m/autos_bw/auto048_rep14.png	896	0.4480
1133	data_12m/autos_bw/auto048_rep15.png	896	0.4480
1134	data_12m/autos_bw/auto048_rep16.png	896	0.4480
1230	data_12m/autos_bw/auto033_rep06.png	948	0.4740
1306	data_12m/autos_bw/auto138_rep01.png	1033	0.5165
1307	data_12m/autos_bw/auto138_rep02.png	1033	0.5165
1362	data_12m/autos_bw/auto032_rep11.png	857	0.4285
1363	data_12m/autos_bw/auto032_rep12.png	827	0.4135
1364	data_12m/autos_bw/auto032_rep13.png	827	0.4135
1365	data_12m/autos_bw/auto016_rep10.png	960	0.4800

1389	data_12m/autos_bw/auto096_rep04.png	1013	0.5065
1390	data_12m/autos_bw/auto096_rep05.png	1013	0.5065
1391	data_12m/autos_bw/auto096_rep06.png	1013	0.5065
1439	data_12m/autos_bw/auto002_rep17.png	892	0.4460
1440	data_12m/autos_bw/auto130_rep07.png	980	0.4900
1441	data_12m/autos_bw/auto138_rep03.png	721	0.3605
1442	data_12m/autos_bw/auto130_rep08.png	981	0.4905
1449	data_12m/autos_bw/auto175_rep01.png	1016	0.5080
1450	data_12m/autos_bw/auto175_rep02.png	1016	0.5080
1451	data_12m/autos_bw/auto139_rep11.png	688	0.3440
1462	data_12m/autos_bw/auto045_rep16.png	946	0.4730
1474	data_12m/autos_bw/auto046_rep03.png	967	0.4835
1523	data_12m/autos_bw/auto026_rep17.png	773	0.3865
1524	data_12m/autos_bw/auto026_rep18.png	773	0.3865
1525	data_12m/autos_bw/auto026_rep19.png	773	0.3865
1526	data_12m/autos_bw/auto026_rep20.png	773	0.3865

1.3.1 Visualización de las primeras 5 imágenes fallidas

```
[43]: path = 'results/12m/'

for _, row in fail_df.head(25).iterrows():
    print(f"Auto {row['num_auto']} - errores: {row['bit_errors']}")
    display(Image(filename=row['bw_plate_tx']))
    display(Image(filename=path + row['bw_plate_rx']))
```

Auto 64 - errores: 749



Auto 64 - errores: 792





Auto 64 - errores: 790



Auto 53 - errores: 949



Auto 53 - errores: 945



Auto 45 - errores: 755



Auto 104 - errores: 952



Auto 122 - errores: 672



Auto 106 - errores: 910





Auto 107 - errores: 824



Auto 109 - errores: 773



Auto 61 - errores: 622



Auto 89 - errores: 1032



Auto 127 - errores: 1136



Auto 36 - errores: 810



Auto 93 - errores: 902



Auto 93 - errores: 902



Auto 93 - errores: 902



Auto 95 - errores: 676





Auto 127 - errores: 934



Auto 94 - errores: 805



Auto 56 - errores: 1009



Auto 56 - errores: 1008



Auto 56 - errores: 1009



Auto 105 - errores: 718



```
[44]: # --- Umbral de bits erróneos ---  
thr_err = 15  
  
# --- Agrupar por auto y comprobar si alguna réplica cumple ---  
auto_status = (
```

```

df.groupby('num_auto')['bit_errors']
    .apply(lambda s: (s < thr_err).any())    # True = tiene al menos una buena
)

# Autos sin réplica "buena"
failed_autos = auto_status[~auto_status]    # False significa que falló
num_failed   = failed_autos.size
num_total    = auto_status.size

print(f'Autos sin ninguna réplica con < {thr_err} errores de bit: '
      f'{num_failed} de {num_total}')

# Mostrar la lista (si quieres verla)
display(failed_autos.index.tolist())

```

Autos sin ninguna réplica con < 15 errores de bit: 1 de 179

[138]